

# IBM Data Engineering Course | Coursera

## Summary

---

### Module one

#### Introduction to Linux

#### Introducing Linux and Unix

#### What is an Operating System (OS)?

An OS is software that:

- Manages computer hardware and resources.
- Allows users to interact with hardware to perform tasks.

#### Unix: Origins and Features

- Unix is a family of operating systems, including **Oracle Solaris, FreeBSD, HP-UX, IBM AIX, and Apple macOS**.
- First created in the 1960s at **AT&T Bell Labs** for the PDP-7 computer.
- In the **1970s**, Unix was rewritten in **C**, making it portable across different hardware.
- **BSD (Berkeley Software Distribution)**, developed in the late 1970s, extended Unix's capabilities.
- **macOS** is derived from BSD Unix.

#### Linux: A Unix-Like System

- Linux is a free, open-source alternative to Unix.
- Developed by **Linus Torvalds** in **1991** as an open-source Unix kernel.
- Combined with **GNU tools**, it became a fully functional OS.

#### Key Features of Linux

- **Free & Open-Source:** Source code is publicly available.
- **Secure:** Extensive community review enhances security.
- **Multi-User:** Supports simultaneous user access.
- **Multitasking:** Runs multiple applications at once.
- **Portable:** Works on various devices, from desktops to servers.

#### Common Use Cases for Linux

- **Mobile devices:** Android OS uses a Linux-based kernel.
- **Supercomputers:** Powering high-performance computing clusters.

- **Cloud & Data Centers:** Running web servers, databases, and enterprise applications.
- **Personal Computers:** Many use Linux distributions like Ubuntu.

## Key Takeaways

- Unix originated in the 1960s and was later extended by BSD.
- Linux emerged in the 1980s-90s as an open-source Unix-like OS.
- Linux is portable, multi-user, and multitasking.
- It is widely used in smartphones, supercomputers, and cloud computing.

---

## Linux Distributions

A **Linux distribution (distro)** is a specific version of the Linux operating system that:

- Uses the **Linux kernel** as its core.
- Includes a unique set of **default utilities** (commands and applications).
- Has its own **graphical user interface (GUI)**.
- Supports a specific set of **shell commands**.
- Offers different levels of **support** (community-driven or enterprise-backed).
- Can be a **Long-Term Support (LTS)** release or a **rolling release** (continuous updates).

## Common Linux Distributions and Their Use Cases

### 1. Debian

- One of the earliest distros (released in 1993).
- **Stable, reliable, and fully open-source.**
- Supports multiple hardware architectures.
- **Highly regarded in server environments.**
- Largest **community-driven** Linux distro.

### 2. Ubuntu (*Debian-based*)

- Released in **2004**, developed by **Canonical**.
- **Three editions:**
  - **Desktop** (for PCs, laptops, and workstations).
  - **Server** (for file servers and cloud computing).
  - **Core** (for IoT devices).

### 3. Red Hat Enterprise Linux (RHEL)

- A **core Linux distro**, not derived from another.
- Developed by **Red Hat (an IBM subsidiary)**.
- **Enterprise-focused**, designed for business use.

- Stable and secure, with commercial support.

#### 4. Fedora (*Sponsored by Red Hat*)

- **Secure and stable**, supports many architectures.
- Rapidly developed with **frequent updates**.
- Used by **Red Hat as a testing ground** for RHEL.

#### 5. SUSE Linux Enterprise (SLE)

- **Two editions:**
  - **SLES** (Server edition).
  - **SLED** (Desktop edition).
- Supports multiple architectures (e.g., ARM for Raspberry Pi).
- Uses **SUSE Package Hub** for additional software.
- Developed by the **German company SUSE**.

#### 6. Arch Linux

- **Highly customizable**, designed for advanced users.
- Users configure their own system from the ground up.
- Uses a **rolling release model** (latest software updates).
- **Not focused on stability**, requires strong Linux knowledge.

### Key Takeaways

- Linux distros differ in **user interface, shell utilities, and support model**.
- Each distro is designed for a **specific audience** (servers, enterprises, desktops, etc.).
- **Debian** is widely used in servers for its **stability**.
- **RHEL** is an **enterprise-grade** Linux used in business environments.
- **SUSE Linux Enterprise** supports diverse **hardware architectures**.

---

## Overview of Linux Architecture

A **Linux system** is structured into five key layers:

1. **User Interface (UI)**
  - Allows users to interact with the system via **keyboard, mouse, or GUI**.
  - GUI-based Linux systems function similarly to Microsoft Windows.
2. **Application Layer**
  - Contains **system daemons, shells, user applications, and system tools**.
  - Examples:
    - **System tools** (e.g., compilers, programming languages).
    - **Shells** (command-line interfaces like Bash).
    - **User apps** (e.g., browsers, text editors, music players).

### 3. Operating System (OS)

- Ensures **system stability** through:
  - **Job scheduling** and managing system resources.
  - **File management.**
  - **Error detection and prevention.**

### 4. Kernel (*Core of the OS*)

- Lowest-level software that interacts with hardware.
- **Boots with the system and remains in memory.**
- Handles **system calls** (communication between apps and hardware).
- Performs four key jobs:
  1. **Memory management.**
  2. **Process management.**
  3. **Managing device drivers.**
  4. **Ensuring system security.**

### 5. Hardware Layer

- Includes **physical components** such as:
  - **CPU** (performs calculations).
  - **RAM** (temporary data storage for running applications).
  - **Persistent storage** (HDDs, SSDs).
  - **USB devices** (keyboard, mouse, drives).

---

## Linux Filesystem Overview

The **Linux filesystem** organizes files and directories in a **tree-like hierarchy**:

- **Root directory (/)** → The topmost directory in Linux.
- **Key system directories:**
  - `/bin` → Stores **binary executable files** (commands/programs).
  - `/usr` → Contains **user-installed applications**.
  - `/home` → User's **personal directory** for files and settings.
  - `/boot` → Holds **system boot files** (vital for startup).
  - `/media` → Temporary mount point for **USB drives, CDs, etc.**

---

## Key Takeaways

- **Linux consists of five main layers:** UI, Applications, OS, Kernel, and Hardware.
  - **The kernel is the core** that enables apps to interact with hardware.
  - **The Linux filesystem** follows a **tree-like structure**, with `/` as the root directory.
  - **System files are categorized into specific directories**, ensuring **efficient organization and security**.
-

## Linux Terminal Overview

The **shell** is an OS-level application that **interprets user commands**.

- It allows users to:
  - Move and copy files.
  - Read and write data.
  - Extract and filter data.
  - Search for files.
- Common shell types:
  - **Bash** (Bourne Again Shell).
  - **Zsh** (Z Shell).

### What is a Linux Terminal?

- The **terminal** is a user interface where users input commands.
- It relays commands to the shell, which interacts with the **OS and kernel**.
- Example:
- `python myprogram.py`
  - The shell interprets and executes the command.
  - The program prints "**Hello, World!**" to the terminal.

### How Commands Work in Linux

1. **User** enters a command in the terminal.
  2. **Terminal** sends the command to the shell.
  3. **Shell** forwards the request to the **kernel**.
  4. **Kernel** translates the command for the hardware.
  5. **Hardware** executes the task.
  6. **Results** return through the shell to the terminal.
- 

## Navigating the Linux Filesystem Using Terminal

### Command Line Basics

- **Command line:** The area where you enter commands.
- **Command prompt:** The cursor position indicating where text will appear.
- **Current working directory:** The location where the shell looks for files and executes commands.

### Filesystem Path Symbols

- `/` → Root directory.
- `~` → User's home directory.
- `.` → Current directory.
- `..` → Parent directory.

## Changing Directories (`cd` command)

- **Go to the root directory**
  - `cd /`
- **Move into a directory (e.g., `bin`)**
  - `cd bin`
- **Navigate to the home directory**
  - `cd ~`
- **Move up one level**
  - `cd ..`
- **Move up two levels**
  - `cd ../..`
- **Navigate directly to a specific path**
  - `cd /home/me/Documents`

## Listing Directory Contents (`ls` command)

- **List files in the current directory**
    - `ls`
  - **Run `ls` from the `/bin` directory**
    - `./ls`
  - **Run `ls` from any location**
    - `ls`
- 

## Key Takeaways

- **The Linux shell** processes user commands.
  - **The terminal** is an interface for sending commands to the shell.
  - **The `cd` command** is used to navigate the filesystem.
  - **Understanding paths** (absolute and relative) helps in directory navigation.
  - The **`ls`** command lists the contents of a directory.
  - The **`pwd`** command prints the path name to the present working directory
- 

## Creating and Editing Text Files

### Types of Text Editors

#### ◆ Command-line text editors:

- **GNU nano** – A simple and user-friendly text editor.
- **vi** – A traditional Unix-based text editor.
- **vim** – An enhanced version of vi with powerful features.

### ◆ GUI-based text editors:

- **gedit** – A modern, easy-to-use text editor from the GNOME environment.
- **emacs** – One of the oldest open-source text editors, available in both GUI and command-line modes.

---

## Using gedit

To open a file in gedit:

```
gedit filename.txt
```

Features:

- ✓ Undo/redo
- ✓ Search and replace
- ✓ Syntax highlighting
- ✓ File browser integration

---

## Using nano

To open a file in nano:

```
nano filename.txt
```

Basic commands:

- **Navigation:** Use arrow keys.
- **Search:** CTRL + W
- **Save changes:** CTRL + O
- **Exit:** CTRL + X

---

## Using vim

To open a file in vim:

```
vim filename.txt
```

**Modes in vim:**

- **Insert mode:** Press `i` to start typing.
- **Command mode:** Press `ESC` to execute commands.

Basic commands:

```
:w    # Save file
:q    # Quit
:q!   # Quit without saving
:wq   # Save and quit
```

---

## Key Takeaways

- ✓ Linux text editors are divided into GUI-based and command-line editors.
  - ✓ Nano is simple, while Vim is powerful but requires learning.
  - ✓ Gedit offers a user-friendly GUI experience.
- 

## Installing Software and Updates

### Understanding Packages and Package Managers

- **Packages:** Archive files containing software installation or updates.
  - **Package Managers:** Tools that manage downloading, installing, and updating packages.
  - **Types of Packages:**
    - `.deb` → Used by Debian-based distros (**Debian, Ubuntu, Mint**).
    - `.rpm` → Used by Red Hat-based distros (**CentOS/RHEL, Fedora, openSUSE**).
  - **Converting between package formats:**
    - Use the **alien** tool:
    - `sudo alien package.rpm` # Convert RPM to DEB
    - `sudo alien -r package.deb` # Convert DEB to RPM
- 

### GUI-Based Package Managers

#### ✓ Update Manager (Debian-based distros)

- Checks for updates daily (security updates auto-installed).
- To manually update:
  - Open **Update Manager** → Select updates → Click **Install Updates** → Enter password (if prompted).

#### ✓ PackageKit (RPM-based distros)

- Displays a **starburst icon** when updates are available.
  - To update:
    - Click the starburst icon → Select updates → Click **Install Updates** → Enter password.
-

## Command-Line Package Managers

### ◆ APT (for Debian-based systems)

```
sudo apt update # Refresh package list
sudo apt upgrade # Upgrade all installed packages
sudo apt install package_name # Install a specific package
```

### ◆ YUM (for RPM-based systems)

```
sudo yum update # Update all packages
sudo yum install package_name # Install a specific package
```

---

## Installing Software with Package Managers

- **APT (Deb-based):**
- `sudo apt install package_name`
- **YUM (RPM-based):**
- `sudo yum install package_name`

### 💡 Python Package Managers:

Many software applications use package managers like **pip** or **conda** for managing Python environments.

Example: Installing `pandas` using `pip`:

```
pip install pandas
```

---

## Key Takeaways

- ✓ `.deb` (Debian-based) and `.rpm` (Red Hat-based) are the main package formats in Linux.
  - ✓ **Update Manager** and **PackageKit** are GUI-based package managers for **Debian** and **RPM** distros.
  - ✓ **APT** and **YUM** are command-line package managers for **Debian** and **RPM** distros.
  - ✓ Software installation and updates can be done via GUI or command-line tools.
- 

## Lesson's Summary:

- In the 1980s, GNU was developed at MIT. GNU stands for “GNU’s not Unix” and was made as a free, open source set of the existing Unix system tools. And in 1991, Linus Torvalds developed a free, open source version of the Unix kernel called Linux.
- Linux is widely used today in mobile devices, desktops, supercomputers, data centers, and cloud servers.

- Linux distributions (also known as distros) differ by their UIs, shell applications, and how the OS is supported and built.
  - The design of a distro is catered toward its specific audience and/or use case. Popular Linux distributions include Red Hat Enterprise Linux (RHEL), Debian, Ubuntu, Suse (SLES, SLED, OpenSuse), Fedora, Mint, and Arch.
  - The Linux system consists of five key layers: the UI, application, OS, kernel, and hardware. The user interface enables users to interact with applications. Applications enable users to perform tasks within the system. The operating system runs on top of the kernel and is vital for system health and stability, and the kernel is the lowest-level software that enables applications to interact with hardware. Hardware includes all the physical or electronic components of your PC.
  - The Linux filesystem is a tree-like structure consisting of all directories and files on the system.
  - A Linux shell is an OS-level application that you can use to enter commands. You use a terminal to send commands to the shell, and you can use the `cd` command to navigate around your Linux filesystem.
  - You can use a variety of command-line or GUI-based text editors such as GNU nano, vim, vi, and gedit.
  - .deb and .rpm are distinct file types used by package managers in Linux operating systems.
  - You can use GUI-based and command-line package managers to update and install software on Linux systems.
- 

## Informational, Navigational, & Management Commands

### Overview of Common Linux Shell Commands

#### What is a Shell?

- A **shell** is a command-line interface for Unix-like operating systems.
  - It **interprets commands**, runs programs, and enables access to files and utilities.
  - It is also a **scripting language**, allowing task automation.
  - **Common Shells:**
    - **Bash** (default in most Linux systems).
    - Other shells: **SH, KSH, TCSH, ZSH, fish**.
  - To check your default shell:
  - `printenv SHELL`
  - To switch to Bash:
  - `bash`
- 

#### Applications of Shell Commands

- **Getting system information**
- **Navigating and managing files and directories**

- **Printing file/string contents**
  - **File compression and archiving**
  - **Network operations**
  - **System performance monitoring**
  - **Running batch jobs (e.g., ETL operations)**
- 

## **Common Shell Commands**

### **❏Getting System Information**

```
whoami # Displays the current username
id     # Shows user and group IDs
uname  # Prints OS name
ps     # Lists running processes
top    # Displays CPU/memory usage
df     # Shows filesystem usage
man <command> # Opens manual page for a command
date   # Prints current date/time
```

### **❏File Management**

```
cp file1 file2 # Copy file
mv file1 new_location # Move or rename file
rm file1 # Delete file
touch newfile # Create empty file / update timestamp
chmod 755 file # Change file permissions
wc file # Count lines, words, and characters in a file
grep "pattern" file # Search for lines matching a pattern
```

### **❏Directory Navigation & Management**

```
ls # List files in the current directory
find . -name "file*" # Search for files matching a pattern
pwd # Show current directory path
mkdir newdir # Create a new directory
cd /path/to/dir # Change directory
rmdir dir # Remove an empty directory
```

### **❏Displaying File Contents**

```
cat file # Display entire file contents
more file # View file one page at a time
head -n 10 file # Show first 10 lines
tail -n 10 file # Show last 10 lines
echo "Hello" # Print text or variable value
```

### **❏File Compression & Archiving**

```
tar -cvf archive.tar files/ # Create an archive
zip archive.zip file1 file2 # Compress files
unzip archive.zip # Extract zip archive
```

## 🔌 Networking Commands

```
hostname # Show system hostname
ping google.com # Send packets and check response
ifconfig # Display network interfaces
curl https://example.com # Fetch URL contents
wget https://example.com/file.zip # Download a file
```

---

## Running Linux on Windows

- **Dual boot** (Linux installed on a separate partition).
  - **Virtual Machine** (Linux runs inside Windows).
  - **CygWin** (Linux-like environment for Windows).
  - **WSL (Windows Subsystem for Linux)** (Runs Linux directly in Windows).
- 

## Key Takeaways

- ✅ A **shell** is an interface for executing commands and scripts.
  - ✅ **Shell commands** are used for **file navigation, file management, networking, and system monitoring**.
  - ✅ The **echo** command prints strings or variables, while **cat** and **tail** display file contents.
  - ✅ **curl** and **wget** retrieve files from the internet.
  - ✅ Linux can run on Windows via **dual boot, VM, CygWin, or WSL**.
- 

## Informational Commands

### User and OS Information

- `whoami` → Displays the current username
- `id` → Returns the user ID (UID) and group ID (GID)
  - `id -u` → Shows only the user ID
  - `id -un` → Displays the username corresponding to the UID
- `uname` → Provides OS details
  - `uname -s` → Displays the OS name
  - `uname -r` → Shows the OS version
  - `uname -v` → Provides detailed version info

### Checking Disk Usage

- `df -h ~` → Displays disk usage of the home directory in a human-readable format
- `df -h` → Shows disk space usage for all mounted filesystems

## Monitoring Running Processes

- `ps -e` → Lists all active processes with their process IDs
- `top` → Provides a real-time view of running processes and system performance
  - `top -n 3` → Displays the top three resource-consuming processes

## Printing Strings, Variables, and Dates

- `echo "text"` → Prints the provided text
- `echo $VARIABLE` → Displays the value of a variable (e.g., `echo $PATH`)
- `date` → Shows the current system date and time
  - `date "+%j %Y"` → Prints the day of the year and the current year

## Accessing Manuals for Commands

- `man <command>` → Displays the manual page for any command
  - Example: `man id` → Shows the manual for the `id` command

## Key Takeaways

- Use `whoami` and `id` to get user info
- Retrieve OS details with `uname`
- Check disk usage with `df`
- Monitor system processes using `ps` and `top`
- Print text, variable values, and formatted dates with `echo` and `date`
- Use `man` to access documentation for any Linux command

---

## File and Directory Navigation Commands

### Objectives

By the end of this lesson, you will be able to:

- List a directory's contents using the `ls` command
- Navigate directories with the `cd` command
- Differentiate between relative and absolute paths
- Locate files using the `find` command

### Listing Directory Contents (`ls`)

- `ls` → Lists files and directories in the current location
- `ls Downloads` → Lists contents of the *Downloads* folder
- `ls -l` → Displays detailed file information, including:
  - File permissions
  - Owner
  - Last modified date

## Checking the Current Directory (`pwd`)

- `pwd` → Prints the full path of the current working directory

## Changing Directories (`cd`)

- `cd Documents` → Moves into the *Documents* folder (relative path)
- `cd ..` → Moves up one directory (to the parent folder)
- `cd ~` → Moves directly to the home directory (absolute path)
- `cd /Users/me/Documents/Notes` → Moves to the *Notes* directory using an absolute path

## Finding Files (`find`)

- `find . -name "a.txt"` → Searches for *a.txt* in the current directory and subdirectories
- `find . -iname "a.txt"` → Case-insensitive search for *a.txt*

## Key Takeaways

- `ls` lists files and directories
- `pwd` shows the current directory
- `cd` allows movement between directories using relative or absolute paths
- `find` helps locate files based on specific criteria

---

## File and Directory Management Commands

By the end of this lesson, you will be able to:

- Create and delete files and directories
- Copy and move files and directories
- Manage file permissions to make them executable

## Creating and Deleting Files & Directories

- `mkdir test` → Creates a directory named *test*
- `touch file.txt` → Creates an empty file named *file.txt*
- `rm file1` → Deletes *file1*
- `rm -r folder1` → Recursively deletes *folder1* and its contents
- `rmdir test` → Removes an empty directory

## Copying and Moving Files & Directories

- `cp notes.txt Documents` → Copies *notes.txt* to the *Documents* folder
- `cp -r Documents Docs_copy` → Recursively copies *Documents* to *Docs\_copy*
- `mv my_script.sh Scripts` → Moves *my\_script.sh* to the *Scripts* folder

- `mv Notes Scripts Documents` → Moves *Notes* and *Scripts* folders into *Documents*

### Managing File Permissions (`chmod`)

- `ls -l my_script.sh` → Displays file permissions
- `chmod +x my_script.sh` → Grants execute permission
- Running the script now works as expected

### Key Takeaways

- `touch` creates files and updates modification timestamps
- `mkdir` creates directories, and `rmdir` removes empty ones
- `cp` and `mv` allow copying, moving, and renaming files and directories
- `chmod` modifies file permissions to enable execution

---

## Summary

- `pwd` to get the location of your present working directory
- `ls` to list the files and directories within a directory
- `mkdir` to create a new directory
- `cd` to change your present working directory
- `touch` to create a new file
- `find` to search for and locate files
- `rm` to remove a file
- `mv` to rename or move a file
- `cp` to copy a file

---

## Security: Managing File Permissions and Ownership

- There are three possible levels of file ownership in Linux - user, group, and other - which determine who can read, write to, and execute a file
  - You can use the `ls -l` command to view file and directory permissions
  - You can change permissions on a file by using the `chmod` command
-

## Working with Text Files, Networking & Archiving Commands

### Viewing File Content

#### Objectives

By the end of this lesson, you will be able to:

- View file content using different commands
- Navigate large files efficiently
- Count lines, words, and characters in a file

#### Commands for Viewing File Content

1. **cat (Concatenate and display file content)**
  - `cat numbers.txt` → Prints entire file to standard output
2. **more (Paginated view for large files)**
  - `more numbers.txt` → Displays content page by page
  - Press Space → Move to the next page
  - Press q → Quit
3. **head (View the beginning of a file)**
  - `head numbers.txt` → Shows first 10 lines
  - `head -n 3 numbers.txt` → Shows first 3 lines
4. **tail (View the end of a file)**
  - `tail numbers.txt` → Shows last 10 lines
  - `tail -n 3 numbers.txt` → Shows last 3 lines

#### Counting Lines, Words, and Characters (wc)

- `wc pets.txt` → Displays **7 7 28** `pets.txt`
  - **7 lines**
  - **7 words**
  - **28 characters (including newline characters)**
- `wc -l pets.txt` → Displays **line count**
- `wc -w pets.txt` → Displays **word count**
- `wc -c pets.txt` → Displays **character count**

#### Key Takeaways

- `cat` prints the full content but is impractical for large files
  - `more` allows paginated viewing
  - `head` and `tail` extract specific parts of a file
  - `wc` provides line, word, and character statistics
-

## Useful Commands for Wrangling Text Files

### Useful Commands for Wrangling Text Files

#### Objectives

By the end of this lesson, you will be able to:

- Sort file contents alphabetically or in reverse order
  - Remove repeated lines
  - Extract lines containing a specific pattern
  - Extract slices and fields from each line
  - Merge lines from multiple files
- 

#### Sorting and Removing Duplicates

##### 1. Sorting Files (**sort**)

- `sort pets.txt` → Sorts lines **alphabetically**
- `sort -r pets.txt` → Sorts lines in **reverse** order

##### 2. Removing Duplicates (**uniq**)

- `uniq pets.txt` → Removes **consecutive** duplicate lines
  - `sort pets.txt | uniq` → Ensures removal of all duplicates
- 

#### Searching for Patterns (**grep**)

##### 3. Extracting Specific Lines (**grep**)

- `grep "ch" people.txt` → Shows lines **containing** "ch"
  - `grep -i "ch" people.txt` → Case-insensitive search (matches "Charles")
  - `grep -v "ch" people.txt` → Shows lines **excluding** "ch"
- 

#### Extracting Fields and Slices (**cut**)

##### 4. Cutting Fields and Characters (**cut**)

- `cut -c 2-9 people.txt` → Extracts **characters 2 to 9** from each line
  - `cut -d " " -f 2 people.txt` → Extracts the **second field** (last name)
- 

#### Merging Multiple Files (**paste**)

## 5. Merging Files (**paste**)

- o `paste first.txt last.txt yob.txt` → Merges files into a **tab-separated** table
  - o `paste -d "," first.txt last.txt yob.txt` → Uses **comma** as a separator
- 

## Key Takeaways

- `sort` organizes text **alphabetically** or in **reverse order**
  - `uniq` removes only **consecutive** duplicate lines (use `sort` first)
  - `grep` extracts lines **matching a pattern**, with case-insensitive and exclusion options
  - `cut` extracts **specific characters or fields** based on delimiters
  - `paste` merges multiple files into a **table format** with customizable delimiters
- 

## Networking Commands

### Examining Network Configuration

1. **Checking Host Name (`hostname`)**
    - o `hostname` → Displays the **host name** of your machine
    - o `hostname -s` → Shows the **short** version (without domain suffix)
    - o `hostname -i` → Retrieves the **IP address** of your host
  2. **Displaying Network Interfaces (`ifconfig`)**
    - o `ifconfig` → Lists all **network interfaces**, including:
      - IP & MAC addresses
      - Data transmission stats
    - o `ifconfig eth0` → Displays **details of a specific interface** (e.g., Ethernet)
- 

### Evaluating Connection Stability

3. **Testing Network Connectivity (`ping`)**
    - o `ping google.com` → Sends continuous ICMP packets to **check connectivity**
    - o `ping -c 5 google.com` → Sends **5 ping requests** and prints results
    - o Output includes:
      - Target's **IP address**
      - **Round-trip times** (min, avg, max, std deviation)
      - **Packet loss percentage**
-

## Retrieving Data from a URL

### 4. Downloading Data (curl)

- o `curl www.google.com` → Fetches the **HTML content** of the page
- o `curl -o google.txt www.google.com` → Saves the output to a **file**
- o `head google.txt` → Verifies **downloaded content**

### 5. Retrieving Files (wget)

- o `wget URL` → Downloads the file at the **specified URL**
  - o `wget -O custom_name.txt URL` → Saves with a **custom filename**
  - o `head -12 file.txt` → Displays the **first 12 lines** of the file
- 

## Key Takeaways

- `hostname` retrieves the **system's hostname and IP**
  - `ifconfig` displays **network interface details**
  - `ping` checks **connectivity and stability** to a server
  - `curl` fetches **data from URLs**
  - `wget` downloads **files from URLs** with advanced options
- 

## File Archiving and Compression Commands

### 🗄️ Archiving vs. Compression

- **Archiving:** Combines multiple files into a single file for storage and backup.
- **Compression:** Reduces file size by eliminating redundancy, improving storage efficiency and transfer speeds.
- These processes are often used together.

### 📦 Creating and Extracting Archives

- `tar -cf archive.tar directory/` → Creates an archive (tarball).
- `tar -xf archive.tar` → Extracts files from a tar archive.
- `tar -czf archive.tar.gz directory/` → Creates a compressed tarball with gzip.
- `tar -xzf archive.tar.gz` → Extracts and decompresses a tar.gz file.

### 📁 Checking Archive Contents

- `tar -tf archive.tar` → Lists files inside an archive.

### 📦 ZIP Compression & Extraction

- `zip -r archive.zip directory/` → Compresses and archives using ZIP.
- `unzip archive.zip` → Extracts files from a ZIP archive.

## 💡 Key Takeaways

- **Archiving** makes file management easier and enhances portability.
- **Compression** reduces file size to save space and improve transfer speeds.
- `tar` is mainly used for archiving, while `zip` compresses before bundling files.
- Both `tar` and `zip` allow easy extraction of files when needed.

---

## Module Summary:

- A shell is an interactive user interface. You use shell commands to navigate and work with files and directories.
  - The `curl` and `wget` commands display and download files from URLs, and the `cat` and `tail` commands display file contents.
  - You can get user information with the `whoami` and `id` commands, and get operating system information using the `uname` command. You can check system disk usage using the `df` command and monitor processes and resource usage with `ps` and `top`. Print string or variable value using `echo`, print and extract information about the date with the `date` command, and read the manual for any command using `man`.
  - `ls` lists all files and directories within a specified directory tree and `cd` navigates between directories. The `find` command finds files in your directories.
  - Relative paths are relative to your current working directory, while absolute paths stand independently.
  - You can create files and directories with the `touch` and `mkdir` commands, delete them with `rm` and `rmdir`, and copy and move them `cp` and `mv`.
  - The `cat`, `more`, `head`, and `tail` commands allow you to sort and view file contents or view only a certain number of lines. Determine line, word, and character counts with `wc`.
  - You can use `sort` to view the lines of a file alphanumerically and `uniq` to remove repeated lines from your view. `grep` gets the lines of a file that match your desired criteria, and `cut` extracts slices and fields from lines. You can merge lines from different files using `paste`.
  - `hostname` and `ip` allow you to view the network configuration. You can test a network connection using `ping` and send and receive data using `curl` and `wget`.
  - Compression preserves storage space, speeds data transfer, and reduces system load.
  - `zip` compresses files and folders prior to archiving them. `tar` archives and compresses files and directories into a tarball. `unzip` unpacks and decompresses a zipped archive, and `tar` can also decompress and unpack a `tar.gz` archive.
-

# Introduction Shell Scripting

## Shell Scripting Basics

### What is a Script?

- A **script** is a list of commands executed by an interpreter at runtime (not compiled).
- Used for **automation**, including ETL jobs, file backups, system administration, and application integration.

### Shebang (!) and Interpreter Directive

- The **first line** in a script specifies the interpreter:
  - `#!/bin/sh` → Uses the Bourne shell.
  - `#!/bin/bash` → Uses Bash.
  - `#!/usr/bin/env python3` → Uses Python.
- The shebang ensures the correct interpreter is used to execute the script.

### Creating and Running a Shell Script

1. **Create a script file:**
2. `touch hello_world.sh`
3. **Add shebang and script commands:**
4. `echo "#!/bin/bash" >> hello_world.sh`
5. `echo "echo Hello World" >> hello_world.sh`
6. **Make the script executable:**
7. `chmod +x hello_world.sh`
8. **Run the script:**
9. `./hello_world.sh`

### File Permissions

- Check file permissions using:
- `ls -l hello_world.sh`
- The `rw`x flags represent **read (r)**, **write (w)**, and **execute (x)** permissions for the owner, group, and others.
- `chmod +x` makes the script **executable**.

### Key Takeaways

- **Shell scripts** automate tasks and can invoke other programs.
  - **Shebang (!)** determines the script's interpreter.
  - **Scripts are interpreted**, not compiled, making them easier to develop but slower to execute than compiled programs.
  - **Permissions** must be set correctly to execute a script.
-

## Filters, Pipes, and Variables

### Filters in Shell

- **Filters** are shell commands that **take input** from **standard input** (keyboard) and **return output** to **standard output** (terminal).
- Examples: `wc`, `cat`, `more`, `head`, `sort`, `grep`.
- Filters **transform** input data into output data.

### Pipes (|)

- Pipes **connect** multiple filter commands, allowing the **output of one command** to be the **input for another**.
- Syntax:
  - `command1 | command2 | command3`
- Example: Reverse sorting directory contents:
  - `ls | sort -r`

### Shell Variables

- **Shell variables** exist **only in the shell** where they are defined.
- List all shell variables:
  - `set`
- Define a new shell variable:
  - `GREETINGS="Hello"`
- Access a variable:
  - `echo $GREETINGS`
- Multiple variables:
  - `AUDIENCE="World"`
  - `echo "$GREETINGS $AUDIENCE"`
- Remove a variable:
  - `unset AUDIENCE`

### Environment Variables

- **Extended scope:** Environment variables **persist** in **child processes**.
- Convert a shell variable into an environment variable:
  - `export GREETINGS`
- List environment variables:
  - `env`
- Search for a specific variable:
  - `env | grep GREE`

### Key Takeaways

- **Filters** transform data from input to output.
- The **pipe operator (|)** chains commands together.
- **Shell variables** are local, while **environment variables** persist across child processes.

- Use `export` to convert a shell variable into an **environment variable**.
- 

## Useful Features of the Bash Shell

### ◆ Metacharacters in Bash

- **Metacharacters:** Special characters with meaning in the shell.
- `#` → Comments (ignored by the shell).
- `;` → Separates multiple commands on the same line.
- `*` → Matches any number of characters in filenames.
- `ls /bin/ba*` # Lists all files in /bin starting with 'ba'
- `?` → Matches a single character in filenames.
- `ls /bin/bas?` # Matches 'bash' but not 'base'

### ◆ Quoting in Bash

- **Backslash (\)** → Escapes a single character.
- `echo \$1` # Prints '\$1' instead of treating it as a variable
- **Double quotes (")** → Interprets metacharacters except `$`, `\`, and ```.
- `var="world"`
- `echo "Hello $var"` # Output: Hello world
- **Single quotes (')** → Treats everything literally.
- `echo '$var'` # Output: \$var

### ◆ Input/Output (I/O) Redirection

- **Redirect output to a file (> overwrites, >> appends)**
- `echo "Hello" > file.txt` # Overwrites file.txt
- `echo "World" >> file.txt` # Appends to file.txt
- **Redirect errors (2> to overwrite, 2>> to append)**
- `ls nonexistentfile 2> error.log` # Redirects error output
- **Redirect input (<)**
- `wc -l < file.txt` # Counts lines in file.txt

### ◆ Command Substitution

- Store command output in a variable:
- `here=$(pwd)` # Using `$()`
- `here=`pwd`` # Using backticks
- `echo $here` # Prints current directory

### ◆ Command Line Arguments

- Used to pass values to a Bash script.
- `./MyBashScript.sh arg1 arg2`
- Inside the script:
- `echo "First argument: $1"`
- `echo "Second argument: $2"`

## ◆ Bash Execution Modes

- **Batch Mode (Sequential Execution)**
- `command1; command2` # Runs `command1` first, then `command2`
- **Concurrent Mode (Parallel Execution)**
- `command1 & command2` # Runs `command1` in the background, `command2` in the foreground

## 💡 Key Takeaways

- **Metacharacters** define special operations in Bash.
- **Quoting** controls how special characters are interpreted.
- **I/O redirection** allows saving output, handling errors, and reading input from files.
- **Command substitution** assigns command outputs to variables.
- **Command-line arguments** pass values to scripts.
- **Concurrent mode** enables parallel execution of commands.

## Examples of Bash Shell Features

- Metacharacters such as `#` `;` `*` and `?` are characters that the shell interprets with special meanings.
- Quoting allows you to ensure any special characters, spaces, or other metacharacters are interpreted literally by the shell.
- Input/output redirection redirects a program's input or output to/from a file.
- Command substitution allows you to use the output of a command as an argument for another command.
- Command line arguments can be used to pass information to a shell script.

## Introduction to Advanced Bash Scripting

- Conditional statements can be used to run commands based on whether a specified condition is true.
- Logical operators do true/false comparisons.
- Arithmetic operators perform basic arithmetic calculations.
- You can create list-like arrays and access their individual elements.
- `for` loops execute operations repeatedly, based on a looping index.

---

## Scheduling Jobs Using Cron

### ◆ What is Cron?

- **Cron:** A Linux/Unix utility for scheduling jobs to run automatically.
- **Crond:** The background service (daemon) that executes cron jobs.
- **Crontab (Cron Table):** A file that stores scheduled jobs and their execution times.

## ◆ Creating and Managing Cron Jobs

- **Editing crontab:**
- `crontab -e` # Opens the default text editor to edit cron jobs
- **Listing existing cron jobs:**
- `crontab -l` # Displays all scheduled cron jobs
- **Removing a cron job:**
  - Open `crontab -e`, delete the specific line, and save.

## ◆ Cron Syntax

Each cron job follows this format:

|        |        |        |        |       |                                     |
|--------|--------|--------|--------|-------|-------------------------------------|
| *      | *      | *      | *      | *     | command-to-execute                  |
|        |        |        |        |       |                                     |
|        |        |        |        | +---- | Day of the week (0 - 6, Sunday = 0) |
|        |        |        | +----- |       | Month (1 - 12)                      |
|        |        | +----- |        |       | Day of the month (1 - 31)           |
|        | +----- |        |        |       | Hour (0 - 23)                       |
| +----- |        |        |        |       | Minute (0 - 59)                     |

- \* → Wildcard (any value).

## ◆ Examples of Cron Jobs

### ☐ Run a script daily at midnight:

```
0 0 * * * /path/to/load_data.sh
```

### ☐ Backup every Sunday at 2 AM:

```
0 2 * * 0 /path/to/backup.sh
```

### ☐ Append date to 'sundays.txt' every Sunday at 15:30:

```
30 15 * * 0 date >> /path/to/sundays.txt
```

## 💡 Key Takeaways

- **Cron automates recurring tasks** using a predefined schedule.
  - **Crond runs cron jobs** at their scheduled times.
  - **Crontab files store job schedules** and can be edited using `crontab -e`.
  - **Cron syntax consists of 5 fields** for time specification.
  - **Use `crontab -l` to list jobs** and `crontab -e` to modify them.
-

## Lesson's Summary

- A shell script is a program that begins with a 'shebang' directive and is used to run commands and programs. Scripting languages are interpreted rather than compiled.
  - Filters are shell commands. The pipe operator `|` allows you to chain filter commands.
  - Shell variables can be assigned values with `=` and listed using `set`. Environment variables are shell variables with extended scope, and you can list them with `env`.
  - Metacharacters are special characters that have meaning to the shell.
  - Quoting specifies whether the shell should interpret special characters as metacharacters or 'escape' them.
  - Input/Output, or I/O redirection, refers to a set of features used for redirecting.
  - You can use command substitution to replace a command with its output.
  - Command line arguments provide a way to pass arguments to a shell script.
  - In concurrent mode, multiple commands can run simultaneously.
  - You can schedule cron jobs to run periodically at selected times. `m h dom mon dow command` is the cron job syntax.
  - You can edit cron jobs by running `crontab -e`, and `crontab -l` lists all cron jobs in the cron table.
-